FIR Benchmark Results

Sequential FPGA vs Parallel FPGA vs x86 CPU

Benchmark elements

We compare execution organization for the same FIR task:

- Sequential FPGA: one MAC datapath reused across taps
- Parallel FPGA: spatial replication, 1 sample/clock until timing limits appear
- x86 CPU: instruction-driven execution, high frequency but higher cycles/sample

Results

- Seq FPGA throughput falls from 7.69 to 3.03 MS/s as taps rise 8→32
- Parallel FPGA holds 100 MS/s at 8 and 16 taps; 32 taps fails timing at 100 MHz
- x86 CPU reaches 132.06 MS/s on the 8-tap benchmark

Topic: temporal reuse vs spatial replication.

Measured data summary

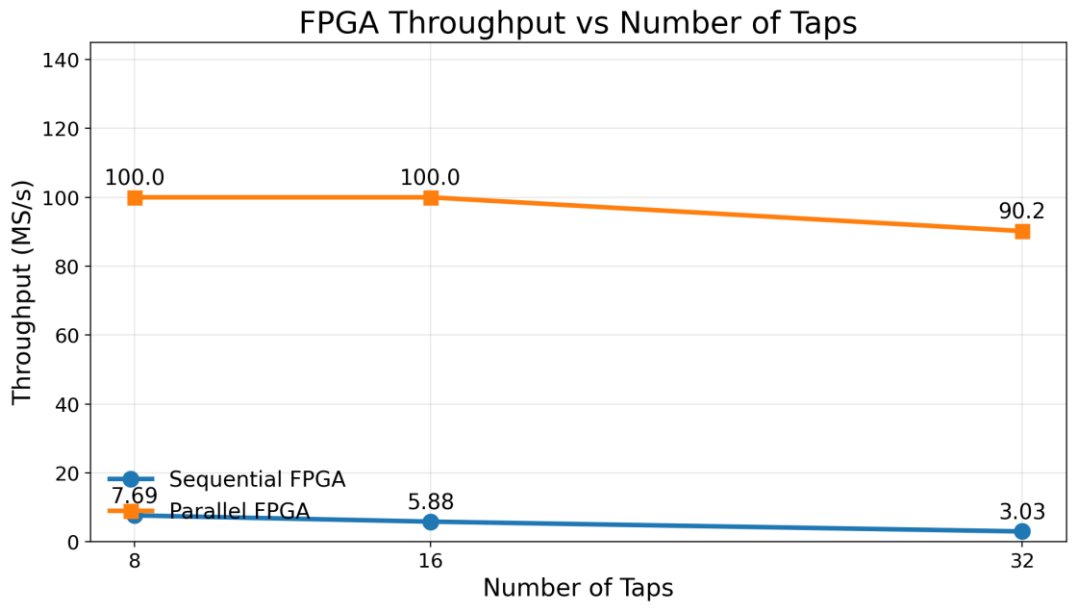
Clock target = 100 MHz for FPGA; x86 data for 8 taps

Design	Taps	Cycles/sample	Latency (ns)	Throughput (MS/s)	LUTs	FFs	WNS (ns)
Seq FPGA	8	13	90	7.69	63	95	4.285
Seq FPGA	16	17	170	5.88	58	160	5.599
Seq FPGA	32	33	330	3.03	90	289	5.186
Par FPGA	8	1	10	100	95	169	2.296
Par FPGA	16	1	10	100	225	281	0.265
Par FPGA	32	1	10	90.2*	422	537	-1.090
x86 CPU	8	26.2	—	132.06	—	—	—

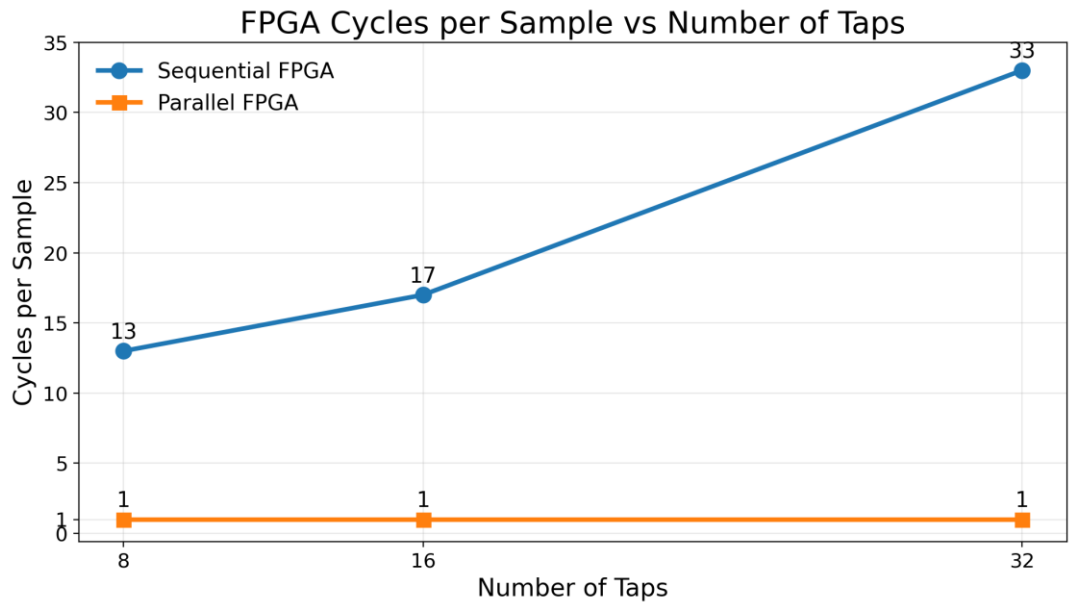
**32-tap parallel throughput adjusted to estimated achievable rate because the design failed 100 MHz timing after implementation.*

Throughput = Clock Frequency / Cycles per Sample. Estimated Fmax for Parallel-32 taps ≈ 1 / (10 ns + 1.090 ns) ≈ 90.2 MHz.

Core architecture : Scaling behavior across tap count



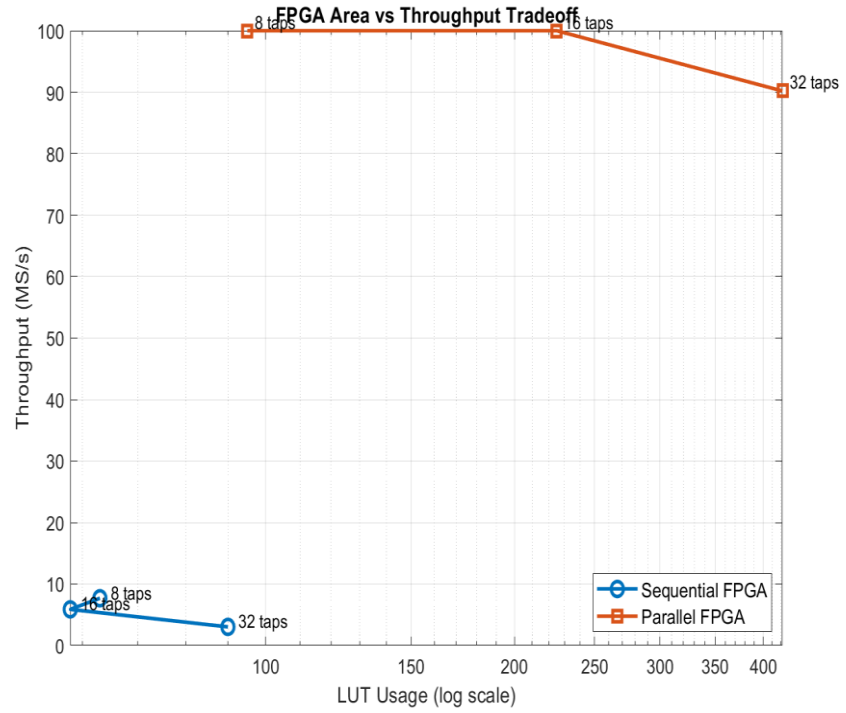
Sequential throughput falls as taps grow; parallel stays flat until timing pressure appears.



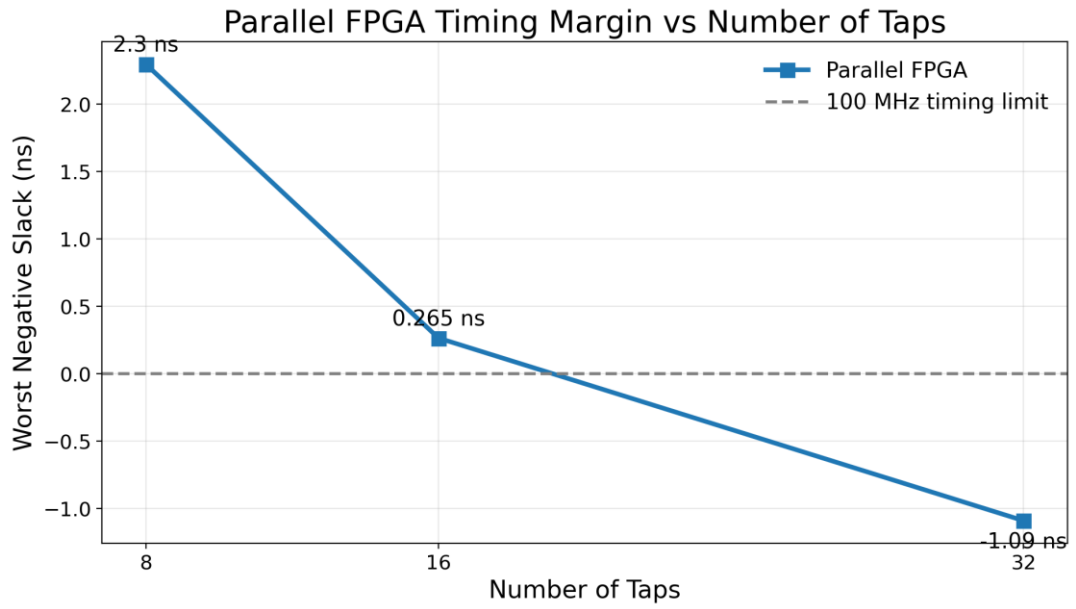
Sequential work grows with tap count, while parallel stays at 1 cycle/sample.

Area and timing tradeoffs

Parallel FIR preserves throughput by spending much more hardware and timing margin



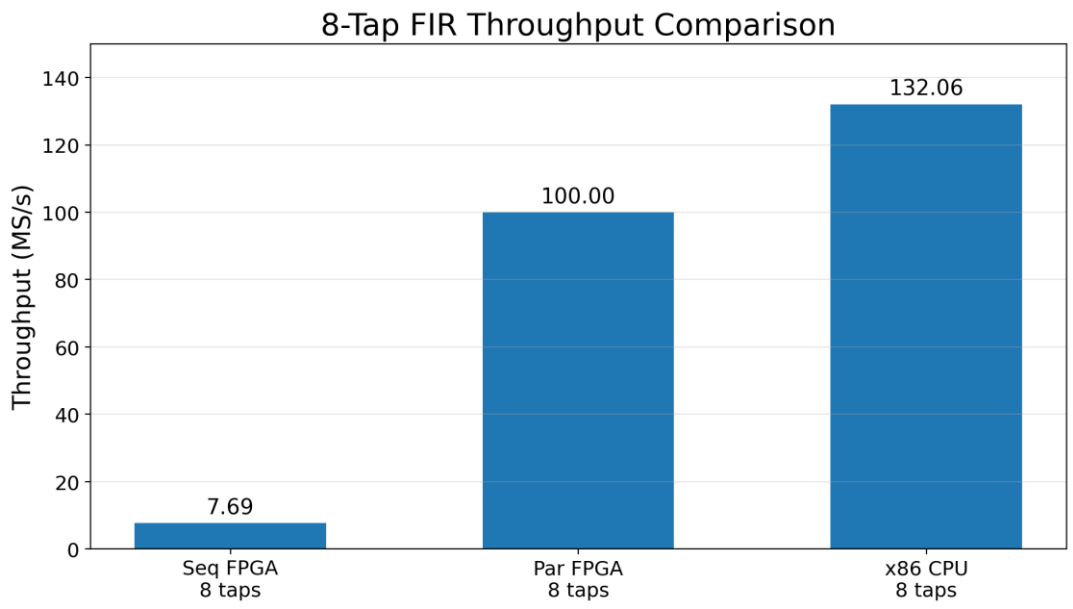
The parallel architecture climbs from 95 to 422 LUTs to hold high throughput.



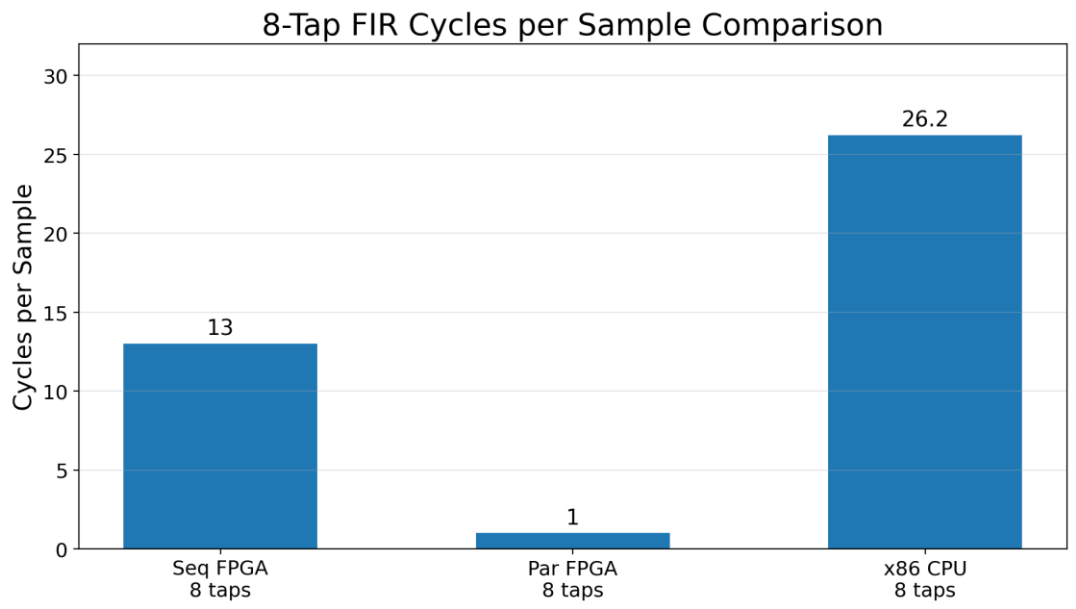
At 32 taps, WNS becomes negative, so 100 MHz is no longer physically achievable without deeper pipelining.

8-tap cross-architecture comparison

CPU-vs-FPGA with all three implementations using 8 taps



x86 leads raw throughput at 8 taps, because of much higher frequency.



Per-sample work is far more efficient on the parallel FPGA than on the CPU.

Conclusion

- Sequential FPGA scales in time: more taps directly increase cycles/sample and reduce throughput.
- Parallel FPGA scales in hardware: it keeps 1 sample/clock at 8 and 16 taps, but timing margin collapses by 32 taps.
- x86 CPU wins 8-tap raw throughput (132.06 MS/s), but uses 26.2 cycles/sample versus 1 cycle/sample on the parallel FPGA.

conclusion:

Parallel FPGA achieves dramatically lower cycles/sample, while sequential FPGA uses far less area; x86 trades much higher clock frequency for worse per-sample efficiency.